Decoding Tanner codes on bipartite two-sided vertex expanders

Zihe Zhou Ningzi Wu Zitan Chen

Abstract

We study decoding of Tanner codes defined on (c, d)-regular bipartite two-sided vertex expanders, where each right vertex enforces a binary linear code of length d and minimum distance d_0. We present a deterministic linear-time decoding algorithm that corrects a linear number of adversarial errors under conditions relating the left expansion ratio δ_{L}, the right expansion ratio δ_{R}, and the local distance d_0. Our results show that strong right-to-left expansion can relax the required left-to-right expansion. In particular, when the right expansion ratio is $\delta_{\mathrm{R}} > 1 - 2/d$ and $d_0 \geq 3$, our algorithm corrects a linear number of errors in linear time under the condition $\delta_{\mathrm{L}} > 1/(d_0 - 1)$, improving previously known sufficient trade-offs between the left expansion ratio and the local distance, at the expense of requiring strong expansion from the constraint side.

1 Introduction

A Tanner code [Tan81] is a binary linear error-correcting code specified by a sparse bipartite graph $\mathcal{G} = (V \cup U, E)$ together with a local binary linear code $\mathcal{C}_0$ of length d and minimum distance d_0. The graph $\mathcal{G}$ connects $n = |V|$ variable nodes on the left to $m = |U|$ constraint nodes on the right. Moreover, every constraint node $u \in U$ is assumed to have degree d and associated with a copy of the local code $\mathcal{C}_0$. A vector $\boldsymbol{x} \in \mathbb{F}_2^V$ is a codeword of the Tanner code if, for every constraint node $u \in U$, the restriction of $\boldsymbol{x}$ to the neighbors of u belongs to $\mathcal{C}_0$.

Because Tanner codes are defined by local constraints imposed through the underlying bipartite graph $\mathcal{G}$, their error-correcting performance is naturally tied to the *vertex expansion* properties of $\mathcal{G}$. The seminal work of Sipser and Spielman [SS96] showed that this intuition is indeed correct. They considered families of bipartite graphs $\mathcal{G}$ in which every variable node has degree c and which satisfy the following *one-sided* vertex expansion property: for every subset $X \subseteq V$ with $|X| \leq \alpha n$, the number of neighboring nodes of X on the right is at least $\delta_{\mathrm{L}} c|X|$. Here α controls the range of expansion, while δ_{L} measures the *expansion ratio*. Such graphs are called $(c, d, \alpha, \delta_{\mathrm{L}})$ bipartite one-sided vertex expanders. One of the main results of [SS96] shows that, when the local code $\mathcal{C}_0$ is the single parity-check code and the expansion ratio satisfies $\delta_{\mathrm{L}} > 3/4$, the resulting Tanner code can be decoded in $O(n)$ time from $\Omega(n)$ adversarial errors using a bit-flipping algorithm.

Since the work of Sipser and Spielman [SS96], there has been a broad line of research on decoding Tanner codes from a linear number of adversarial errors in linear time. The work of Viderman [Vid13] also considered the case where the local code $\mathcal{C}_0$ is the single parity-check code, and improved

Z. Zhou and N. Wu are with School of Science and Engineering, The Chinese University of Hong Kong (Shenzhen), Shenzhen, Guangdong, 518172, P. R. China (Email: {zihezhou,ningziwu}@link.cuhk.edu.cn). They contributed equally to this work.

Z. Chen is with School of Science and Engineering, Future Networks of Intelligence Institute, The Chinese University of Hong Kong (Shenzhen), Shenzhen, Guangdong, 518172, P. R. China (Email: chenztan@cuhk.edu.cn).

the lower bound on the expansion ratio that suffices for correcting $\Omega(n)$ errors in $O(n)$ time to

$$\delta_{\mathrm{L}} > \frac{2}{3} - \frac{1}{6c}.$$

The key idea in [Vid13] is an erasure-based decoding perspective, in which errors are first identified and then treated as erasures. It was also shown in [Vid13] that Tanner codes defined by certain $(c, d, \alpha, 1/2)$ bipartite expanders have distance two, and therefore cannot correct $\Omega(n)$ errors. Thus, when the local code is the single parity-check code, the condition $\delta_{\mathrm{L}} > 1/2$ is necessary for a uniform guarantee that *all* such Tanner codes correct $\Omega(n)$ errors in $O(n)$ time.[1]

The works of [SS96] and [Vid13] focus on the case in which the local distance is $d_0 = 2$. Subsequent work generalized the local code to arbitrary binary linear codes with minimum distance $d_0 \geq 2$ and investigated the trade-off between the expansion ratio δ_{L} and the local distance d_0 required for Tanner codes to correct $\Omega(n)$ errors in $O(n)$ time. Chilappagari *et al.* [Chi+10] showed that, for $d_0 \geq 3$, the condition

$$\delta_{\mathrm{L}} > \frac{d_0 + 5}{2(d_0 + 3)}$$

is sufficient for this decoding guarantee. They left open whether such a guarantee is possible for every $\delta_{\mathrm{L}} > 0$. Dowling and Gao [DG17] answered this question affirmatively by proving that, for every $\delta_{\mathrm{L}} > 0$, it suffices to take $d_0 = \Omega(c\delta_{\mathrm{L}}^{-2})$. More recently, Shen *et al.* [She+25] showed that the condition $\delta_{\mathrm{L}} > 3/d_0$ is sufficient for correcting $\Omega(n)$ errors in $O(n)$ time. Using an approach similar to that of [Vid13], they also showed that the condition $\delta_{\mathrm{L}} > 1/d_0$ is necessary for a uniform guarantee over all Tanner codes. The work of Zhou and Guo [ZG25] further improved the sufficient condition to $\delta_{\mathrm{L}} > 2/d_0$. Moreover, by generalizing Viderman's erasure-based decoding framework, [ZG25] showed that

$$\delta_{\mathrm{L}} > \frac{d_0}{2d_0 - 1}$$

also suffices. This improves on the requirement $\delta_{\mathrm{L}} > 2/d_0$ for $d_0 = 2, 3$ and gives the best known sufficient condition on the expansion ratio for $d_0 = 3$. For $d_0 = 2$, however, the bound gives $\delta_{\mathrm{L}} > 2/3$, which is still weaker than Viderman's bound. Thus, despite substantial progress, the trade-off between the expansion ratio δ_{L} and the local distance d_0 is not yet fully understood.

The results discussed above all concern Tanner codes on bipartite one-sided vertex expanders. In this work, we study Tanner codes defined on bipartite *two-sided* vertex expanders, and investigate how the interaction between the expansion properties and the local distance affects the existence of linear-time decoding algorithms that correct a linear number of errors. A two-sided vertex expander is a bipartite graph that satisfies both left-to-right and right-to-left vertex expansion properties. More precisely, in addition to the left-to-right expansion condition above, we require that for every $Y \subseteq U$ with $|Y| \leq \beta m$, the number of neighboring nodes of Y on the left is at least $\delta_{\mathrm{R}} d|Y|$, where β controls the range of right-to-left expansion and δ_{R} is the corresponding expansion ratio. Our main result can be stated informally as follows.

Theorem 1.1 (Informal version of Theorem 3.9)**.** *Let $\mathcal{C}_0$ be a binary linear code of length d and minimum distance d_0, and let $\mathcal{G}$ be a bipartite two-sided vertex expander with left expansion ratio δ_{L} and right expansion ratio δ_{R}. If $d_0 - d(1 - \delta_{\mathrm{R}}) > 1$ and*

$$\delta_{\mathrm{L}} > \frac{1}{\lfloor d_0 - d(1 - \delta_{\mathrm{R}}) \rfloor + 1},$$

[1] However, this does not rule out the possibility that for $\delta_{\mathrm{L}} \leq 1/2$, there exist some Tanner codes that can correct $\Omega(n)$ errors in $O(n)$ time, with the local code being the single parity-check code.

then the resulting Tanner code admits a deterministic decoding algorithm that corrects $\Omega(n)$ adversarial errors in $O(n)$ time.

Theorem 1.1 shows that, for a fixed local distance d_0, stronger right-to-left expansion permits a weaker requirement on the left expansion ratio while still guaranteeing good decoding performance. In particular, Theorem 3.9 yields the following corollary.

Corollary 1.2 (Corollary of Theorem 3.9)**.** *For $d_0 \geq 3$ and $\delta_{\mathrm{R}} > 1 - 2/d$, if*

$$\delta_{\mathrm{L}} > \frac{1}{d_0 - 1},$$

then the resulting Tanner code admits a deterministic decoding algorithm that corrects $\Omega(n)$ adversarial errors in $O(n)$ time.

Since $\frac{1}{d_0-1} < 2/d_0$ for all $d_0 \geq 3$, and $\frac{1}{d_0-1} < \frac{d_0}{2d_0-1}$ for $d_0 = 3$, Corollary 1.2 improves the trade-off between the *left* expansion ratio and the local distance obtained in [ZG25], at the expense of requiring strong right-to-left expansion. Our result therefore relies on the availability of good two-sided vertex expanders. We next briefly review the relevant literature on bipartite vertex expanders.

Brief review of bipartite vertex expanders. It is well known that the existence of bipartite vertex expanders can be established by the probabilistic method; see, for example, [SS96, Theorem 26] and [HLW06, Theorem 4.16]. Explicit constructions of bipartite vertex expanders with one-sided *lossless* vertex expansion, meaning $\delta_{\mathrm{L}} \geq 1 - \epsilon$ for arbitrarily small $\epsilon > 0$, were given in the celebrated work of Capalbo *et al.* [Cap+02]. Simpler explicit constructions were later presented by Golowich [Gol24] and Cohen *et al.* [CRT23]. These constructions, however, are one-sided expanders. Explicit constructions of strong two-sided vertex expanders were obtained only recently.[2] A significant breakthrough was achieved by Hsieh *et al.* [Hsi+25a], who constructed the first explicit two-sided vertex expanders with expansion ratio exceeding one half on both sides. Building on this result, a follow-up work by Hsieh *et al.* [Hsi+25b] obtained the first explicit two-sided lossless vertex expanders.

In this paper, we assume the availability of bipartite vertex expanders with the desired expansion properties and use them to construct Tanner codes that admit linear-time decoding algorithms correcting a linear number of adversarial errors. Before turning to the technical details, we first provide an overview of the main ideas behind our decoding algorithm.

1.1 Technical overview

Our decoding algorithm follows an erasure-based strategy, similar to what is adopted in [Vid13] and [ZG25]. Instead of correcting errors directly, the decoder first constructs a partially decoded word $\tilde{\boldsymbol{y}}$ together with a set $\bar{\mathcal{E}}_\ell \subseteq \{1, \ldots, n\}$ from the received word $\boldsymbol{y} \in \mathbb{F}_2^n$, where ℓ is a parameter, such that every coordinate of $\tilde{\boldsymbol{y}}$ outside $\bar{\mathcal{E}}_\ell$ agrees with the transmitted codeword. The decoder then erases the coordinates in $\bar{\mathcal{E}}_\ell$ and applies a peeling erasure decoder. By Lemma 2.3, if $|\bar{\mathcal{E}}_\ell| \leq \alpha n$ and $\delta_{\mathrm{L}} d_0 > 1$, then left expansion guarantees that the peeling decoder recovers the transmitted codeword. Thus, the main challenge is to construct, in linear time, a set $\bar{\mathcal{E}}_\ell$ that is both large enough to contain all erroneous coordinates and small enough for erasure decoding to succeed.

[2] We note that a one-sided lossless vertex expander is also a two-sided lossless vertex expander for certain expansion parameters; see Proposition 3.10 and Remark 3.11.

The construction of $\bar{\mathcal{E}}_\ell$ has two stages. In the first stage, each constraint node locally decodes its neighboring variable nodes using a minimum-distance decoder for the local code $\mathcal{C}_0$. Each variable node then receives one locally decoded value from each of its neighboring constraints. If these local values disagree, the variable node is declared suspicious and is placed in an initial erasure set $\mathcal{E}$. Equivalently, we form a partially decoded word $\tilde{\boldsymbol{y}}$ by keeping the unanimous local value when all neighboring constraints agree, and by placing an erasure symbol otherwise. The set $\mathcal{E}$ is formed by the erased coordinates in $\tilde{\boldsymbol{y}}$. The suspicious coordinates detected in this way are called *visible* errors.

However, some errors may remain undetected by this local voting step. It is possible that all neighboring constraint nodes assign the same value to a variable node, but that unanimous value is still incorrect. Such errors are called *invisible* errors. Thus, the second stage of the decoder enlarges the initial set $\mathcal{E}$ so as to include all invisible errors as well.

The key observation is that invisible errors can be detected indirectly through the visible ones. Suppose the locally decoded word at a constraint node u differs from the true local codeword. Since both words belong to the local code $\mathcal{C}_0$, their difference is a nonzero codeword of $\mathcal{C}_0$, and hence its support has size at least d_0. Therefore, local decoding errors occur on supports of nonzero local codewords. If many coordinates in such a support have already been marked suspicious, then the remaining coordinates in the same support are natural candidates for invisible errors.

This motivates the so-called ℓ-completion procedure. Starting from $\mathcal{E}$, we repeatedly apply the following rule: whenever the support S of a nonzero local codeword has at most ℓ coordinates outside the current set, we add all coordinates of S to the current set. The process stops when no such support remains, and the resulting set is denoted by $\bar{\mathcal{E}}_\ell$. The parameter $\ell \in \{1, \dots, d_0 - 1\}$ controls how aggressively the decoder propagates the initial erasures. Larger ℓ makes it easier to capture invisible errors, but it also risks increasing the size of the completed set.

The left expansion property is used to prove that the completion procedure captures all invisible errors. Indeed, if some invisible errors remained outside $\bar{\mathcal{E}}_\ell$, then each neighboring constraint of these remaining errors would have to contain at least $\ell + 1$ such errors; otherwise the completion rule would have added them. Counting edges and using left expansion then gives a contradiction when $\delta_{\mathrm{L}}(\ell + 1) > 1$. This is formalized in Theorem 3.5.

The right expansion property is used for the complementary purpose: it prevents the completion process from growing too large. Intuitively, each active constraint node that adds new coordinates during the completion process must already have at least $d_0 - \ell$ coordinates in the current set, which serve as witnesses. Right-to-left expansion limits how much these witnesses can overlap across many active constraints. As a result, if the initial set $\mathcal{E}$ is small, then its completion $\bar{\mathcal{E}}_\ell$ remains small as well. The precise bound is given in Theorem 3.7.

Combining these ingredients yields the decoding guarantee in Theorem 3.9. Lemma 3.3 shows that the initial erasure set $\mathcal{E}$ has size at most proportional to the number of errors. Theorem 3.7 then bounds the size of $\bar{\mathcal{E}}_\ell$, while Theorem 3.5 ensures that all invisible errors are included in $\bar{\mathcal{E}}_\ell$. Thus, outside $\bar{\mathcal{E}}_\ell$, the partially decoded word $\tilde{\boldsymbol{y}}$ agrees with the transmitted codeword. If $|\bar{\mathcal{E}}_\ell| \le \alpha n$, the peeling decoder recovers the transmitted codeword from $\tilde{\boldsymbol{y}}$ with erasures on $\bar{\mathcal{E}}_\ell$.

Finally, the entire procedure runs in linear time. For constant c and d, local decoding at each constraint node takes $O(1)$ time, and there are $m = O(n)$ constraint nodes. Thus, the initial local decoding step and the construction of $\mathcal{E}$ take $O(n)$ time. Moreover, because d is constant, each constraint node has only $O(1)$ supports of nonzero local codewords. Using a queue-based implementation, each support is processed at most once and each coordinate is inserted into the completed set at most once, so $\bar{\mathcal{E}}_\ell$ can be constructed in $O(n)$ time; see Proposition 3.1. The peeling decoder also runs in $O(n)$ time by Remark 2.4. Therefore, the overall decoding algorithm runs in $O(n)$ time.

Paper outline. The rest of the paper is organized as follows. Section 2 introduces the notation, basic definitions, and preliminary results used throughout the paper. Section 3 is devoted to the decoding algorithm and its analysis. Finally, Section 4 concludes with several open problems and future directions.

2 Preliminaries

For a positive integer n, let $[n] = \{1, \dots, n\}$. For a vector $\boldsymbol{y} := (y_1, \dots, y_n) \in \mathbb{F}_2^n$, its support is $\mathrm{supp}(\boldsymbol{y}) := \{i \in [n] \colon y_i \neq 0\}$, and its Hamming weight is $\mathrm{wt}(\boldsymbol{y}) := |\mathrm{supp}(\boldsymbol{y})|$. The Hamming distance between two vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{F}_2^n$ is denoted by $d_{\mathrm{H}}(\boldsymbol{x}, \boldsymbol{y}) := \mathrm{wt}(\boldsymbol{x} - \boldsymbol{y})$. For a subset $S \subseteq [n]$, we write $\boldsymbol{y}_S := (y_i)_{i \in S} \in \mathbb{F}_2^S$ for the restriction of $\boldsymbol{y}$ to the coordinates in S.

Let $\mathcal{G} = (V \cup U, E)$ be a bipartite graph. If every vertex $v \in V$ has degree c and every vertex $u \in U$ has degree d, then $\mathcal{G}$ is called (c, d)-regular. For a vertex $w \in V \cup U$, let $N(w) := \{a \in V \cup U \colon (w, a) \in E\}$ be the set of neighboring vertices of w. More generally, for a subset W of either V or U, define $N(W) := \bigcup_{w \in W} N(w)$.

Definition 2.1 (Bipartite vertex expanders)**.** *Let $\mathcal{G} = (V \cup U, E)$ be a (c, d)-regular bipartite graph. If*

$$|N(X)| \geq \delta_{\mathrm{L}} c |X| \tag{1}$$

for every $X \subseteq V$ with $|X| \leq \alpha |V|$, then $\mathcal{G}$ is called a $(c, d, \alpha, \delta_{\mathrm{L}})$ left expander. Similarly, if

$$|N(Y)| \geq \delta_{\mathrm{R}} d |Y| \tag{2}$$

for every $Y \subseteq U$ with $|Y| \leq \beta |U|$, then $\mathcal{G}$ is called a $(c, d, \beta, \delta_{\mathrm{R}})$ right expander. If $\mathcal{G}$ satisfies both (1) and (2), then $\mathcal{G}$ is called a $(c, d, \alpha, \delta_{\mathrm{L}}, \beta, \delta_{\mathrm{R}})$ two-sided expander.

We focus on Tanner codes defined on (c, d)-regular bipartite graphs. Although Tanner codes can be defined over general alphabets, throughout this paper we restrict attention to the binary case.

Definition 2.2 (Tanner codes)**.** *Let $\mathcal{C}_0 \subseteq \mathbb{F}_2^d$ be a binary linear code of length d and minimum distance d_0, and let $\mathcal{G} = (V \cup U, E)$ be a (c, d)-regular bipartite graph. The Tanner code specified by $\mathcal{G}$ and $\mathcal{C}_0$ is defined as*

$$T(\mathcal{G}, \mathcal{C}_0) := \{\boldsymbol{x} \in \mathbb{F}_2^V : \boldsymbol{x}_{N(u)} \in \mathcal{C}_0 \text{ for every } u \in U\},$$

where $\boldsymbol{x}_{N(u)}$ is viewed as a length-d vector according to a fixed ordering of the neighbors of u.

The vertices in V are referred to as *variable nodes* or *code coordinates*, while the vertices in U are referred to as *constraint nodes*. Throughout the paper, we identify V with $[n]$, so that Tanner codewords may also be viewed as vectors in $\mathbb{F}_2^n$. Since $\mathcal{G}$ is (c, d)-regular, we have $|U| = cn/d$. Denote $m := |U|$.

In the sequel, the parameters $c, d, \alpha, \beta, \delta_{\mathrm{L}}, \delta_{\mathrm{R}}$ are treated as constants independent of n, while n is allowed to grow. Thus, when we say that a Tanner code $T(\mathcal{G}, \mathcal{C}_0)$, where $\mathcal{G}$ is a $(c, d, \alpha, \delta_{\mathrm{L}}, \beta, \delta_{\mathrm{R}})$ two-sided expander, can correct $\Omega(n)$ errors in $O(n)$ time, we implicitly refer to an infinite family of such graphs $\{\mathcal{G}_n\}$ with n variable nodes for infinitely many values of n. We also assume that $\mathcal{G}$ is given by adjacency lists, so that the neighbors of each constraint node can be found in time $O(d) = O(1)$.

Erasure decoding of Tanner codes. It is well known that erasure decoding of Tanner codes can be performed by a peeling decoder. We will use the following version of this result.

Lemma 2.3 (Erasure peeling)**.** *Let $\mathcal{G} = (V \cup U, E)$ be a $(c, d, \alpha, \delta_{\mathrm{L}})$ left expander, and let $X \subseteq V$ be the set of erased coordinates of a transmitted Tanner codeword. If $\delta_{\mathrm{L}} d_0 > 1$ and $|X| \le \alpha n$, then all erasures can be recovered by iterative local erasure decoding, as described in Algorithm 1.*

Proof. If $X = \emptyset$, there is nothing to prove. Assume therefore that $X \neq \emptyset$. Suppose, toward a contradiction, that no local erasure-decoding step is possible at any constraint node in $N(X)$. Because a code of minimum distance d_0 can uniquely recover from any pattern of at most $d_0 - 1$ erasures, every constraint node $u \in N(X)$ must then be incident to at least d_0 erased coordinates. Hence,

$$\begin{aligned} \sum_{u \in N(X)} |N(u) \cap X| &\ge d_0 |N(X)| \\ &\ge d_0 \delta_{\mathrm{L}} c |X| \qquad (3) \\ &> c|X|, \qquad (4) \end{aligned}$$

where (3) follows from the expansion property and (4) follows from the assumption $\delta_{\mathrm{L}} d_0 > 1$.

On the other hand, since every erased coordinate has degree c, the same quantity is exactly the number of edges incident to X, namely, $\sum_{u \in N(X)} |N(u) \cap X| = c|X|$, which contradicts (4). Therefore, whenever the current erasure set is nonempty and has size at most αn, there exists a constraint node incident to at least one and at most $d_0 - 1$ erasures. The erased symbols adjacent to this constraint node can be recovered by local erasure decoding.

After each such step, the number of erasures strictly decreases, and the remaining erasure set remains a subset of the original set X, hence still has size at most αn. Repeating the same argument, the peeling process continues until all erasures are recovered. □

Remark 2.4. Under the assumptions of Lemma 2.3, Algorithm 1 recovers the transmitted Tanner codeword in $O(n)$ time. In Algorithm 1, the sets $S(u)$ record the erasures currently present in the neighborhood of each constraint node u. They can be initialized in $O(n)$ time using the adjacency lists of $\mathcal{G}$. Moreover, since d is constant, each invocation of the local erasure decoder for $\mathcal{C}_0$ takes $O(1)$ time. During the algorithm, each erased variable node is recovered and removed exactly once, and this removal requires updating only its $c = O(1)$ neighboring constraint nodes. Finally, each constraint node is enqueued at most once: nodes with at most $d_0 - 1$ erasures are enqueued initially, and any other node can enter the queue only when its number of remaining erasures first drops to $d_0 - 1$. Therefore, the total running time is $O(n)$.

3 Decoding algorithm

In this section, we present our decoding algorithm and analyze its performance. At a high level, the decoder consists of two main steps, as summarized in Algorithm 2. First, we apply Algorithm 4 to identify a set of coordinates that contains all potentially erroneous positions. We then treat these coordinates as erasures and apply the peeling erasure decoder, Algorithm 1, to recover the transmitted codeword. Below we begin with constructing the set containing all the errors.

3.1 Construction of the set that contains all the errors

Let $\boldsymbol{x} = (x_1, \ldots, x_n) \in T(\mathcal{G}, \mathcal{C}_0)$ be the transmitted codeword and let $\boldsymbol{y} = (y_1, \ldots, y_n) \in \mathbb{F}_2^n$ be the received word. We write $\boldsymbol{e} := \boldsymbol{y} - \boldsymbol{x} \in \mathbb{F}_2^n$ for the error vector.

Let $\mathcal{D}_0 \colon \mathbb{F}_2^d \to \mathcal{C}_0$ be a minimum-distance decoder for the local code $\mathcal{C}_0$. For each constraint node $u \in U$, define

$$\boldsymbol{z}_u := \mathcal{D}_0(\boldsymbol{y}_{N(u)}) \in \mathbb{F}_2^{N(u)}$$

to be the locally decoded codeword at u. Strictly speaking, $N(u)$ need not be equal to $[d]$; throughout, we use the fixed ordering of $N(u) \subseteq [n]$ to identify $\mathbb{F}_2^{N(u)}$ with $\mathbb{F}_2^d$ when applying $\mathcal{D}_0$.[3] After decoding, we view the entries of $\boldsymbol{z}_u$ as indexed by the coordinates in $N(u)$. We also assume that ties in $\mathcal{D}_0$ are broken by choosing a fixed minimum-weight coset leader for each coset of $\mathcal{C}_0$.

For each variable node $v \in V$, let $P_v \subseteq \mathbb{F}_2$ denote the set of values assigned to coordinate v by the locally decoded codewords of its neighboring constraint nodes. Writing $\boldsymbol{z}_u = (z_{u,v})_{v \in N(u)}$, define

$$P_v := \{z_{u,v} \colon u \in N(v)\}. \tag{5}$$

Thus, P_v records the collection of local "votes" for the value of coordinate v.

Let $\star$ denote an erasure symbol. We define a partially decoded word $\tilde{\boldsymbol{y}} = (\tilde{y}_1, \ldots, \tilde{y}_n) \in (\mathbb{F}_2 \cup \{\star\})^n$ by

$$\tilde{y}_v = \begin{cases} b, & P_v = \{b\} \text{ for some } b \in \mathbb{F}_2, \\ \star, & P_v = \{0, 1\}, \end{cases} \qquad v \in V. \tag{6}$$

In other words, if all local decoding results agree on the value of coordinate v, then $\tilde{y}_v$ is set to this unanimous value; otherwise, coordinate v is erased. Let

$$\mathcal{E} := \{v \in V \colon \tilde{y}_v = \star\}$$

be the resulting set of erased coordinates.

If $\operatorname{supp}(\boldsymbol{e}) \subseteq \mathcal{E}$ and $\mathcal{E}$ is small enough, then the transmitted codeword can be recovered by erasure decoding. However, it is possible that all local decoding results agree on a value at coordinate v even though this value is incorrect. We call such errors *invisible*. In contrast, errors in $\mathcal{E}$ are called *visible*. Formally, the set of invisible errors is

$$\mathcal{I} := \{v \in V \setminus \mathcal{E} \colon \tilde{y}_v \neq x_v\}.$$

Thus, to reduce the original error-correction problem to erasure decoding, it remains to identify and erase the invisible errors as well.

Our next step is therefore to enlarge the initial erasure set $\mathcal{E}$ to a superset that, under suitable expansion assumptions, contains all error positions. The construction is based on the supports of nonzero codewords of the local code.

For each $u \in U$, write $N(u) = \{v_1^{(u)}, \ldots, v_d^{(u)}\} \subseteq [n]$ where $v_1^{(u)} < \cdots < v_d^{(u)}$. For a codeword $\boldsymbol{c} = (c_1, \ldots, c_d) \in \mathcal{C}_0$, map its support to the neighboring coordinates $N(u)$ by

$$\operatorname{supp}_u(\boldsymbol{c}) := \{v_i^{(u)} \in N(u) \colon i \in \operatorname{supp}(\boldsymbol{c})\} \subseteq V.$$

Define the collection of supports of nonzero local codewords under this mapping by

$$\mathscr{T}_u := \{\operatorname{supp}_u(\boldsymbol{c}) \colon \boldsymbol{c} \in \mathcal{C}_0 \setminus \{0\}\}.$$

[3] For example, one may map the elements in $N(u)$ to the elements in $[d]$ according to their ordering in $N(u) \subseteq [n]$.

We now define the completion procedure. Set $\mathcal{E}^{(0)} = \mathcal{E}$. Fix $\ell \in \{1, \dots, d_0 - 1\}$. For $i = 0, 1, \dots$, define

$$\mathcal{E}^{(i+1)} = \mathcal{E}^{(i)} \cup \Big(\bigcup_{u \in U} \bigcup_{S \in \mathscr{T}_u : |S \setminus \mathcal{E}^{(i)}| \le \ell} S \Big). \tag{7}$$

The sequence of sets $(\mathcal{E}^{(i)})_{i \ge 0}$ is monotone and has a limit since $\mathcal{E}^{(0)} \subseteq \mathcal{E}^{(1)} \subseteq \cdots \subseteq [n]$. We define the *$\ell$-completion* of $\mathcal{E}$ to be this limit:

$$\bar{\mathcal{E}}_\ell := \lim_{i \to \infty} \mathcal{E}^{(i)} = \bigcup_{i \ge 0} \mathcal{E}^{(i)}. \tag{8}$$

In other words, $\bar{\mathcal{E}}_\ell$ is the smallest subset $X \subseteq V$ containing $\mathcal{E}$ with the following property: for every $S \in \bigcup_{u \in U} \mathscr{T}_u$,

$$S \not\subseteq X \implies |S \setminus X| > \ell,$$

or equivalently,

$$|S \setminus X| \le \ell \implies S \subseteq X. \tag{9}$$

Proposition 3.1. *The set $\bar{\mathcal{E}}_\ell$ can be constructed in $O(n)$ time.*

Proof. We first compute the initial set $\mathcal{E}$. Since d is constant, local decoding at each constraint node takes $O(1)$ time. Thus, applying the local decoder to every $u \in U$ and collecting the resulting votes at the variable nodes takes $O(m) = O(n)$ time.

Although the iterative definition (8) is convenient for proving $|\bar{\mathcal{E}}_\ell|$ is small (see Theorem 3.7, recomputing $\mathcal{E}^{(i+1)}$ from scratch at every step would not directly yield a linear-time implementation. Instead, we use the equivalent closure condition (9) and maintain a queue of local supports that are ready to be added.

Let $\mathscr{T} := \bigcup_{u \in U} \mathscr{T}_u$. Since d is constant and each $\mathscr{T}_u$ has size at most $2^d - 1$, we have $|\mathscr{T}| = O(m) = O(n)$. For each $v \in V$, define $\mathrm{inc}(v) := \{S \in \mathscr{T} : v \in S\}$, the collection of local codeword supports containing v. For each $S \in \mathscr{T}$, maintain a counter $\mathrm{absent}(S) := |S \setminus X|$, where X denotes the current set being constructed. Initially, $X = \mathcal{E}$, so $\mathrm{absent}(S) = |S \setminus \mathcal{E}|$.

The algorithm initializes a queue with all sets $S \in \mathscr{T}$ satisfying $\mathrm{absent}(S) \le \ell$. While the queue is nonempty, it removes a set S from the queue and adds all elements of S to X. Whenever a new variable node v is added to X, the counter $\mathrm{absent}(S')$ is decreased by one for every $S' \in \mathrm{inc}(v)$. If after this update some S' now satisfies $\mathrm{absent}(S') \le \ell$, then S' is added to the queue. This algorithm is formally given in Algorithm 3.

Note that, throughout the construction, each node $v \in V$ is added at most once, and each set $S \in \mathscr{T}$ is enqueued at most once. Moreover,

$$\sum_{v \in V} |\mathrm{inc}(v)| = \sum_{S \in \mathscr{T}} |S| = O(n),$$

because $|\mathscr{T}| = O(n)$ and $|S| \le d = O(1)$ for every $S \in \mathscr{T}$. Therefore, initializing $\mathrm{inc}(v)$ for all $v \in V$ and $\mathrm{absent}(S)$ for all $S \in \mathscr{T}$ takes $O(n)$ time, and all queue updates over the entire execution also take $O(n)$ time. Hence, $\bar{\mathcal{E}}_\ell$ can be constructed in $O(n)$ time. □

The above procedure that produces $\bar{\mathcal{E}}_\ell$ is formalized in Algorithm 4. As we prove below, under the desired expansion assumptions on the underlying graph $\mathcal{G}$, the set $\bar{\mathcal{E}}_\ell$ contains all error positions, i.e., $\mathrm{supp}(\boldsymbol{e}) \subseteq \bar{\mathcal{E}}_\ell$, and is small enough for the peeling erasure decoder to recover the transmitted Tanner codeword.

3.2 Bounds on visible and invisible errors

We begin with a simple consequence of minimum-distance local decoding and (c, d)-regularity. Recall that $\boldsymbol{z}_u = \mathcal{D}_0(\boldsymbol{y}_{N(u)})$ is the local decoding result at constraint node $u \in U$. Define the corresponding local decoding error by

$$\boldsymbol{z}_u^* := \boldsymbol{z}_u - \boldsymbol{x}_{N(u)}.$$

Since both $\boldsymbol{z}_u$ and $\boldsymbol{x}_{N(u)}$ belong to $\mathcal{C}_0$, we have $\boldsymbol{z}_u^* \in \mathcal{C}_0$. The following lemma shows that the total weight of these local decoding errors is controlled by the error vector $\boldsymbol{e}$.

Lemma 3.2. *Assume that $\mathcal{G}$ is a (c, d)-regular bipartite graph. For every $u \in U$,*

$$\mathrm{wt}(\boldsymbol{z}_u^*) \le 2\,\mathrm{wt}(\boldsymbol{e}_{N(u)}).$$

Consequently,

$$\sum_{u \in U} \mathrm{wt}(\boldsymbol{z}_u^*) \le 2c\,\mathrm{wt}(\boldsymbol{e}).$$

Proof. Since $\boldsymbol{z}_u$ is obtained by minimum-distance decoding of $\boldsymbol{y}_{N(u)}$, and $\boldsymbol{x}_{N(u)} \in \mathcal{C}_0$, we have

$$d_{\mathrm{H}}(\boldsymbol{z}_u, \boldsymbol{y}_{N(u)}) \le d_{\mathrm{H}}(\boldsymbol{x}_{N(u)}, \boldsymbol{y}_{N(u)}) = \mathrm{wt}(\boldsymbol{e}_{N(u)}). \tag{10}$$

By the triangle inequality,

$$\begin{aligned} \mathrm{wt}(\boldsymbol{z}_u^*) &= d_{\mathrm{H}}(\boldsymbol{z}_u, \boldsymbol{x}_{N(u)}) \\ &\le d_{\mathrm{H}}(\boldsymbol{z}_u, \boldsymbol{y}_{N(u)}) + d_{\mathrm{H}}(\boldsymbol{y}_{N(u)}, \boldsymbol{x}_{N(u)}) \\ &\le 2\,\mathrm{wt}(\boldsymbol{e}_{N(u)}), \end{aligned}$$

where the last inequality follows from (10). Summing over all $u \in U$ gives

$$\sum_{u \in U} \mathrm{wt}(\boldsymbol{z}_u^*) \le 2 \sum_{u \in U} \mathrm{wt}(\boldsymbol{e}_{N(u)}) = 2c\,\mathrm{wt}(\boldsymbol{e}),$$

where the last equality follows since summing over $u \in U$ counts every coordinate of $\mathrm{supp}(\boldsymbol{e})$ exactly c times. □

The set $\mathcal{E}$ may contain coordinates that are not actually corrupted. The next lemma shows that, nevertheless, its size cannot be much larger than $\mathrm{wt}(\boldsymbol{e})$.

Lemma 3.3 (Visible errors)**.** *Assume that $\mathcal{G}$ is a (c, d)-regular bipartite graph. The set $\mathcal{E}$ of erased coordinates in $\tilde{\boldsymbol{y}}$ satisfies*

$$|\mathcal{E}| \le 2c\,\mathrm{wt}(\boldsymbol{e}).$$

Proof. For each $v \in \mathcal{E}$, the local votes at v are not unanimous. Since the alphabet is binary, at least one neighboring constraint node $u \in N(v)$ must output a value different from x_v, i.e., $z_{u,v}^* = 1$. Thus,

$$|\mathcal{E}| \le \sum_{u \in U} \mathrm{wt}(\boldsymbol{z}_u^*) \le 2c\,\mathrm{wt}(\boldsymbol{e}),$$

where the last inequality follows from Lemma 3.2. □

We next bound the number of invisible errors, namely those coordinates outside $\mathcal{E}$ on which all local decoding results agree, but agree on the wrong value.

Lemma 3.4 (Invisible errors)**.** *Assume that $\mathcal{G}$ is a (c, d)-regular bipartite graph. The set of invisible errors satisfies*

$$|\mathcal{I}| \le 2\,\mathrm{wt}(\boldsymbol{e}).$$

Proof. Let $v \in \mathcal{I}$. By definition, $v \notin \mathcal{E}$, so all local votes at v are unanimous. Since v is an invisible error, this unanimous value differs from x_v. Hence, for every $u \in N(v)$, $z^*_{u,v} = 1$. Since $|N(v)| = c$, each invisible error is counted exactly c times in the total local decoding error weight. Therefore,

$$c|\mathcal{I}| \le \sum_{u \in U} \mathrm{wt}(\boldsymbol{z}^*_u) \le 2c\,\mathrm{wt}(\boldsymbol{e}),$$

where the last inequality follows from Lemma 3.2. Dividing by c on both sides gives $|\mathcal{I}| \le 2\,\mathrm{wt}(\boldsymbol{e})$. □

3.3 All invisible errors are found

We now show that the completion procedure indeed identifies all invisible errors, provided the underlying graph has sufficient left expansion.

Theorem 3.5 (ℓ-completion contains all invisible errors)**.** *Assume that $\mathcal{G}$ is a $(c, d, \alpha, \delta_{\mathrm{L}})$ left expander such that $\delta_{\mathrm{L}}(\ell + 1) > 1$. If $|\mathcal{I}| \le \alpha n$, then*

$$\mathcal{I} \subseteq \bar{\mathcal{E}}_\ell.$$

Proof. We first record two simple observations. First, for every $u \in U$,

$$\mathrm{supp}(\boldsymbol{z}^*_u) \subseteq \mathcal{E} \cup \mathcal{I}. \tag{11}$$

Indeed, if $v \in \mathrm{supp}(\boldsymbol{z}^*_u)$, then $z_{u,v} \neq x_v$. If $v \in \mathcal{E}$, there is nothing to prove. If $v \notin \mathcal{E}$, then all local votes at v agree, so the unanimous value is $\tilde{y}_v = z_{u,v} \neq x_v$, and hence, $v \in \mathcal{I}$.

Second, if $v \in \mathcal{I}$, then for every $u \in N(v)$,

$$v \in \mathrm{supp}(\boldsymbol{z}^*_u). \tag{12}$$

This follows because all local votes at $v \in \mathcal{I}$ are unanimous, and this unanimous value is different from x_v.

We now prove the claim. Let $W := \mathcal{I} \setminus \bar{\mathcal{E}}_\ell$. It suffices to show that $W = \emptyset$. Suppose, toward a contradiction, that W is nonempty. Consider any $u \in N(W)$. Then there exists some $v \in W$ such that $u \in N(v)$. Since $v \in \mathcal{I}$, (12) implies that $v \in \mathrm{supp}(\boldsymbol{z}^*_u)$, and therefore,

$$\mathrm{supp}(\boldsymbol{z}^*_u) \cap W \neq \emptyset.$$

By the definition of W, this implies $\mathrm{supp}(\boldsymbol{z}^*_u) \not\subseteq \bar{\mathcal{E}}_\ell$. On the other hand, by (11) we have $\mathrm{supp}(\boldsymbol{z}^*_u) \setminus \mathcal{E} \subseteq \mathcal{I}$ Noticing $\mathcal{E} \subseteq \bar{\mathcal{E}}_\ell$, we further obtain

$$\mathrm{supp}(\boldsymbol{z}^*_u) \setminus \bar{\mathcal{E}}_\ell \subseteq \mathcal{I} \setminus \bar{\mathcal{E}}_\ell = W.$$

It follows that

$$\mathrm{supp}(\boldsymbol{z}^*_u) \setminus \bar{\mathcal{E}}_\ell = (\mathrm{supp}(\boldsymbol{z}^*_u) \setminus \bar{\mathcal{E}}_\ell) \cap W = \mathrm{supp}(\boldsymbol{z}^*_u) \cap W.$$

Since $\operatorname{supp}(\boldsymbol{z}_u^*) \in \mathscr{T}_u$ and $\operatorname{supp}(\boldsymbol{z}_u^*) \not\subseteq \bar{\mathcal{E}}_\ell$, the definition of $\bar{\mathcal{E}}_\ell$ implies that $|\operatorname{supp}(\boldsymbol{z}_u^*) \setminus \bar{\mathcal{E}}_\ell| > \ell$. Therefore, for $u \in N(W)$,

$$|N(u) \cap W| \geq |\operatorname{supp}(\boldsymbol{z}_u^*) \cap W| \geq \ell + 1.$$

Since $|W| \leq |\mathcal{I}| \leq \alpha n$, left expansion yields

$$c|W| = \sum_{u \in N(W)} |N(u) \cap W| \geq (\ell+1)|N(W)| \geq (\ell+1)\delta_{\mathrm{L}} c|W| > c|W|,$$

which is a contradiction. Therefore, $W = \mathcal{I} \setminus \bar{\mathcal{E}}_\ell = \emptyset$ and $\mathcal{I} \subseteq \bar{\mathcal{E}}_\ell$. □

Remark 3.6. The condition $\delta_{\mathrm{L}}(\ell+1) > 1$ implies $\ell > 1/\delta_{\mathrm{L}} - 1 \geq 0$. In other words, for $\bar{\mathcal{E}}_\ell \supseteq \mathcal{I}$ to hold, it is necessary that $\ell \geq 1$, and thus the initial set $\mathcal{E}$ of erased coordinates in $\tilde{\boldsymbol{y}}$ does not contain all the error coordinates.

3.4 The set containing all errors is small

To make the reduction to erasure decoding work, the set $\bar{\mathcal{E}}_\ell$ must remain small enough for the peeling decoder to succeed. The next theorem shows that right-to-left expansion prevents the completion process from growing too large.

Theorem 3.7 (ℓ-completion remains small)**.** *Assume that $\mathcal{G}$ is a $(c, d, \beta, \delta_{\mathrm{R}})$ right expander such that $s := d(1-\delta_{\mathrm{R}}) < d_0 - \ell$. If*

$$|\mathcal{E}| < (d_0 - \ell - s)\lfloor \beta m \rfloor, \tag{13}$$

then

$$|\bar{\mathcal{E}}_\ell| \leq \frac{d-s}{d_0 - \ell - s}|\mathcal{E}|. \tag{14}$$

Proof. We analyze a sequential process for constructing $\bar{\mathcal{E}}_\ell$. Start with $X = \mathcal{E}$. While there exists a constraint node $u \in U$ and a support $S \in \mathscr{T}_u$ such that $1 \leq |S \setminus X| \leq \ell$, add all coordinates in S to X. The sets in $\bigcup_{u \in U} \mathscr{T}_u$ are processed following fixed total orders on U and on each $\mathscr{T}_u$. This process terminates at the set $\bar{\mathcal{E}}_\ell$ according to (8).

We say that a constraint node u is *active* if it adds at least one new coordinate to X during this process. Note that the same node u may become active multiple times before the process terminates. Let $\bar{\mathcal{A}}_\ell \subseteq U$ denote the set of constraint nodes that become active at least once when the process terminates. We next show that $|\bar{\mathcal{A}}_\ell|$ implies a bound on $|\bar{\mathcal{E}}_\ell|$ via the expansion property of $\mathcal{G}$.

For each active node u, let $Q_u \subseteq N(u)$ be the set of coordinates whose addition to X is caused by u. Then the sets $\{Q_u \colon u \in \bar{\mathcal{A}}_\ell\}$ are disjoint and

$$\bar{\mathcal{E}}_\ell \setminus \mathcal{E} = \bigcup_{u \in \bar{\mathcal{A}}_\ell} Q_u.$$

For each active node u, consider the first time at which u becomes active. At that moment, there exists a support $S_u \in \mathscr{T}_u$ such that $1 \leq |S_u \setminus X| \leq \ell$. Since every set in $\mathscr{T}_u$ is the support of a nonzero codeword of $\mathcal{C}_0$, we have $|S_u| \geq d_0$. Thus, at least $d_0 - \ell$ coordinates of S_u are already present in X before u first becomes active. Choose any such $d_0 - \ell$ coordinates and call them the *witnesses* for u first becoming active.

Let $B_u \subseteq N(u)$ be the union of the witnesses for u and the set Q_u. The witnesses are already present before u first becomes active, whereas the coordinates in Q_u are added to $\bar{\mathcal{E}}_\ell$ by u. Hence, these two sets are disjoint, and therefore

$$|Q_u| = |B_u| - (d_0 - \ell). \tag{15}$$

Moreover, since $B_u \subseteq N(u)$ and $|N(u)| = d$, we have

$$|Q_u| \le d - (d_0 - \ell). \tag{16}$$

We next show that the number of active constraint nodes is small. Suppose, toward a contradiction, that $|\bar{\mathcal{A}}_\ell| \ge \lfloor \beta m \rfloor$. Let $A \subseteq \bar{\mathcal{A}}_\ell$ be the set of the first $\lfloor \beta m \rfloor$ active constraint nodes in the sequential construction process. Then $|A| \le \beta m$, so right expansion gives

$$|N(A)| \ge \delta_{\mathrm{R}} d |A| = (d - s)|A|.$$

Consequently,

$$d|A| - |N(A)| \le s|A|. \tag{17}$$

Since $B_u \subseteq N(u)$ for every $u \in A$, the number of repeated counts in $\sum_{u\in A} |B_u|$ is at most the number of repeated counts in $\sum_{u\in A} |N(u)|$. Therefore, we have

$$\begin{aligned} \sum_{u\in A} |B_u| - \Big| \bigcup_{u\in A} B_u \Big| &\le \sum_{u\in A} |N(u)| - \Big| \bigcup_{u\in A} N(u) \Big| \\ &= d|A| - |N(A)| \\ &\le s|A|. \end{aligned} \tag{18}$$

On the other hand, because A consists of the first $\lfloor \beta m \rfloor$ active nodes, every witness for a node in A either belongs to the initial set $\mathcal{E}$ or was added earlier by another node in A. Therefore,

$$\bigcup_{u\in A} B_u \subseteq \mathcal{E} \cup \bigcup_{u\in A} Q_u.$$

Hence,

$$\Big| \bigcup_{u\in A} B_u \Big| \le |\mathcal{E}| + \sum_{u\in A} |Q_u|.$$

Combining this with (18), we obtain

$$\begin{aligned} s|A| &\ge \sum_{u\in A} |B_u| - \Big| \bigcup_{u\in A} B_u \Big| \\ &\ge \sum_{u\in A} |B_u| - |\mathcal{E}| - \sum_{u\in A} |Q_u| \\ &= \sum_{u\in A} \big(|B_u| - |Q_u| \big) - |\mathcal{E}| \\ &= (d_0 - \ell)|A| - |\mathcal{E}|, \end{aligned} \tag{19}$$

where the last equality follows from (15). Thus,

$$(d_0 - \ell - s)|A| \le |\mathcal{E}|.$$

Since $|A| = \lfloor \beta m \rfloor$, this contradicts (13). Therefore, $|\bar{\mathcal{A}}_\ell| < \lfloor \beta m \rfloor$.

Now take $A = \bar{\mathcal{A}}_\ell$. Since $|\bar{\mathcal{A}}_\ell| < \lfloor \beta m \rfloor \le \beta m$, the same argument above applies and gives $(d_0 - \ell - s)|\bar{\mathcal{A}}_\ell| \le |\mathcal{E}|$. Since $s < d_0 - \ell$, we conclude that

$$|\bar{\mathcal{A}}_\ell| \le \frac{|\mathcal{E}|}{d_0 - \ell - s}.$$

Finally, all the new coordinates added to $\mathcal{E}$ to form $\bar{\mathcal{E}}_\ell$ are precisely those in the disjoint sets Q_u. Using (16), we obtain

$$\begin{aligned} |\bar{\mathcal{E}}_\ell| &= |\mathcal{E}| + \sum_{u \in \bar{\mathcal{A}}_\ell} |Q_u| \\ &\le |\mathcal{E}| + (d - d_0 + \ell)|\bar{\mathcal{A}}_\ell| \\ &\le \Big(1 + \frac{d - d_0 + \ell}{d_0 - \ell - s}\Big)|\mathcal{E}| \\ &= \frac{d - s}{d_0 - \ell - s}|\mathcal{E}|. \end{aligned}$$

This proves (14). □

Remark 3.8. The above proof does not use any special property of $\mathcal{E}$ beyond its size. Thus, the same conclusion holds for any subset of V with size no larger than $(d_0 - \ell - s)\lfloor \beta m \rfloor$.

3.5 Putting it all together

We now combine the previous ingredients to prove the decoding guarantee. Recall that the decoder first constructs the word $\tilde{\boldsymbol{y}}$ and the ℓ-completion set $\bar{\mathcal{E}}_\ell$, and then applies the peeling erasure decoder to $\tilde{\boldsymbol{y}}$ with the coordinates in $\bar{\mathcal{E}}_\ell$ erased.

Theorem 3.9. *Let $\ell \in [d_0 - 1]$, and let $\mathcal{C}_0$ be a binary linear code of length d and minimum distance d_0. Let $\mathcal{G} = (V \cup U, E)$ be a $(c, d, \alpha, \delta_{\mathrm{L}}, \beta, \delta_{\mathrm{R}})$ two-sided expander with $|V| = n$ and $|U| = m$. Set $s := d(1 - \delta_{\mathrm{R}})$. Assume that*

(i) $\delta_{\mathrm{L}}(\ell + 1) > 1$,

(ii) $s < d_0 - \ell$.

Then Algorithm 2 recovers every transmitted codeword $\boldsymbol{x} \in T(\mathcal{G}, \mathcal{C}_0)$ from any received word $\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{e} \in \mathbb{F}_2^n$ satisfying

$$\mathrm{wt}(\boldsymbol{e}) \le \frac{1}{2c} \cdot \min\Big\{ \frac{d_0 - \ell - s}{d - s}\alpha n,\ (d_0 - \ell - s)\lfloor \beta m \rfloor - 1 \Big\}. \tag{20}$$

Moreover, the algorithm runs in $O(n)$ time.

Proof. Let $\mathcal{E}$ be the initial erasure set obtained from the local decoding votes. By Lemma 3.3 and the second bound in (20),

$$|\mathcal{E}| \le 2c\,\mathrm{wt}(\boldsymbol{e}) \le (d_0 - \ell - s)\lfloor \beta m \rfloor - 1 < (d_0 - \ell - s)\lfloor \beta m \rfloor.$$

Theorem 3.7 therefore applies. In particular,

$$|\bar{\mathcal{E}}_\ell| \le \frac{d - s}{d_0 - \ell - s}|\mathcal{E}|.$$

Using Lemma 3.3 again, together with the first bound in (20), we obtain

$$|\bar{\mathcal{E}}_\ell| \le \frac{d-s}{d_0-\ell-s} \cdot 2c\,\mathrm{wt}(\boldsymbol{e}) \le \alpha n.$$

Next, by Lemma 3.4, the set $\mathcal{I}$ of invisible errors satisfies

$$|\mathcal{I}| \le 2\,\mathrm{wt}(\boldsymbol{e}) \le \frac{d_0-\ell-s}{c(d-s)}\alpha n \le \alpha n,$$

where the last inequality follows from $d_0 - \ell \le d$. Since $\delta_{\mathrm{L}}(\ell+1) > 1$, Theorem 3.5 gives $\mathcal{I} \subseteq \bar{\mathcal{E}}_\ell$. It follows that every coordinate outside $\bar{\mathcal{E}}_\ell$ is correct in the word $\tilde{\boldsymbol{y}}$. Thus, after erasing the coordinates in $\bar{\mathcal{E}}_\ell$, the word $\tilde{\boldsymbol{y}}$ agrees with the transmitted codeword $\boldsymbol{x}$ on all unerased coordinates.

It remains to check that the erasure decoder succeeds. Since $\ell \le d_0 - 1$, we have $\delta_{\mathrm{L}} d_0 \ge \delta_{\mathrm{L}}(\ell+1) > 1$. Together with $|\bar{\mathcal{E}}_\ell| \le \alpha n$, Lemma 2.3 implies that the peeling erasure decoder recovers $\boldsymbol{x}$ from $\tilde{\boldsymbol{y}}$ with erasures on $\bar{\mathcal{E}}_\ell$.

Finally, by Proposition 3.1, the completed set $\bar{\mathcal{E}}_\ell$ can be constructed in $O(n)$ time. By Remark 2.4, the peeling decoder also runs in $O(n)$ time. Therefore, the overall running time is $O(n)$. □

3.5.1 Discussion

Theorem 3.9 exhibits a trade-off between the two expansion requirements. Increasing ℓ relaxes the left-expansion condition $\delta_{\mathrm{L}}(\ell+1) > 1$, but it also strengthens the right-expansion requirement $d(1-\delta_{\mathrm{R}}) < d_0 - \ell$ and decreases the decoding radius. Thus, for fixed d_0 and δ_{R}, one should choose ℓ as large as possible subject to the right-expansion condition.

We now derive Corollary 1.2 as a simple consequence of Theorem 3.9.

Proof of Corollary 1.2. Let $\ell = d_0 - 2$. Since $\delta_{\mathrm{R}} > 1 - 2/d$, we have $s = d(1-\delta_{\mathrm{R}}) < 2 = d_0 - \ell$, so Item (ii) of Theorem 3.9 is satisfied. Item (i) becomes $\delta_{\mathrm{L}}(d_0 - 1) > 1$, or equivalently,

$$\delta_{\mathrm{L}} > \frac{1}{d_0 - 1}.$$

Under this condition, Theorem 3.9 gives a linear decoding radius and an $O(n)$-time decoding algorithm. Hence, the Tanner code can correct $\Omega(n)$ adversarial errors in $O(n)$ time. □

We also note that expansion on one side can force expansion on the other side for sufficiently small sets. The following proposition makes this observation precise.

Proposition 3.10. *Let $\mathcal{G} = (V \cup U, E)$ be a $(c, d, \beta, \delta_{\mathrm{R}})$ right expander. Then, for every $\alpha > 0$, $\mathcal{G}$ is also a $(c, d, \alpha, \delta_{\mathrm{L}})$ left expander with*

$$\delta_{\mathrm{L}} \ge \max\left\{\frac{1}{d}, \min\left\{\frac{c-1}{cd(1-\delta_{\mathrm{R}})}, \frac{\beta}{d\alpha}\right\}\right\}. \tag{21}$$

In particular, if $\alpha \le \beta/d$ and $\delta_{\mathrm{R}} \ge 1 - (1+\epsilon)/d$ with $\epsilon \in (0,1)$, then

$$\delta_{\mathrm{L}} \ge \left(1 - \frac{1}{c}\right)\frac{1}{1+\epsilon}.$$

Proof. Let $X \subseteq V$ with $|X| \le \alpha|V|$, and set $Y := N(X) \subseteq U$. We first record the trivial lower bound. By double counting the edges between X and Y, we have $c|X| \le d|Y|$. Thus, $|N(X)| = |Y| \ge c|X|/d$, implying that one may always take $\delta_{\mathrm{L}} \ge 1/d$.

We now derive the stronger bound when $|Y| \le \beta|U|$. Let

$$Z = N(Y) \setminus X.$$

All $c|X|$ edges incident to X are contained in the edge set between X and Y. Since the total number of edges incident to Y is $d|Y|$, the number of edges from Y to Z is $d|Y| - c|X|$. Therefore, $|Z| \le d|Y| - c|X|$. It follows that

$$|N(Y)| = |X| + |Z| \le d|Y| - (c-1)|X|.$$

On the other hand, since $|Y| \le \beta|U|$, right expansion gives

$$|N(Y)| \ge \delta_{\mathrm{R}} d|Y|.$$

Combining the two inequalities yields $\delta_{\mathrm{R}} d|Y| \le d|Y| - (c-1)|X|$. Therefore,

$$|Y| \ge \frac{c-1}{d(1-\delta_{\mathrm{R}})}|X|.$$

Since $|Y| = |N(X)|$, this gives the left-expansion ratio

$$\delta_{\mathrm{L}} \ge \frac{c-1}{cd(1-\delta_{\mathrm{R}})}. \tag{22}$$

If $\alpha \le \beta/d$, then for every $X \subseteq V$ with $|X| \le \alpha|V|$,

$$|Y| = |N(X)| \le c|X| \le c\alpha|V| \le \frac{c\beta}{d}|V| = \beta|U|,$$

where we used $c|V| = d|U|$. Thus, the stronger bound (22) applies to all such X.

It remains to consider $\alpha > \beta/d$. If $|Y| \le \beta|U|$, the same argument as above gives (22). If instead $|Y| \ge \beta|U|$, then, using $|X| \le \alpha|V|$ and $c|V| = d|U|$, we get

$$|N(X)| = |Y| \ge \beta|U| = \frac{\beta c}{d}|V| \ge \frac{\beta c}{d\alpha}|X|.$$

Thus, in this case one may take

$$\delta_{\mathrm{L}} \ge \frac{\beta}{d\alpha}.$$

Combining the two cases with the trivial bound $\delta_{\mathrm{L}} \ge 1/d$ proves (21).

Lastly, if $\alpha \le \beta/d$ and $\delta_{\mathrm{R}} \ge 1 - (1+\epsilon)/d$, then (22) gives $\delta_{\mathrm{L}} \ge (1-1/c)(1+\epsilon)^{-1}$. □

Remark 3.11. By symmetry, one can obtain the analogous statement that sufficiently strong left expansion implies right expansion for small sets, similar to Proposition 3.10. In particular, a $(c, d, \alpha, 1-(1+\epsilon)/c)$ left expander is also a

$$\left(c, d, \alpha, 1 - \frac{1+\epsilon}{c}, \frac{\alpha}{c}, \left(1 - \frac{1}{d}\right)\frac{1}{1+\epsilon}\right)$$

two-sided expander. Thus, when the degrees are large, a one-sided lossless expander already exhibits two-sided lossless expansion, although possibly on a smaller range of set sizes.

Proposition 3.10 shows that, in some parameter regimes, the right-expansion assumption alone already enforces a nontrivial left-expansion guarantee. For example, when $\alpha \leq \beta/d$ and $\delta_R \geq 1 - (1+\epsilon)/d$, the graph automatically has left expansion ratio at least $(1-1/c)(1+\epsilon)^{-1}$. In this regime, Item (i) of Theorem 3.9 may therefore follow directly from the right-expansion assumption.

When α is larger, the implication becomes weaker. For $\beta/d < \alpha < \beta$, a nontrivial right-expansion ratio $\delta_R > 1/c$ already implies a nontrivial lower bound on the left-expansion ratio through (21). Indeed, both terms inside the minimum in (21) are then strictly larger than $1/d$. So the guarantee for δ_L provided by Proposition 3.10 may or may not be sufficient for Item (i).

For $\alpha \geq \beta$, Proposition 3.10 reduces to the trivial guarantee $\delta_L \geq 1/d$. In this case, the left-expansion condition in Theorem 3.9 provides a nontrivial sufficient condition for linear-time correction of a linear number of errors.

4 Open problems

In this work, we presented a decoding algorithm for Tanner codes defined on bipartite two-sided vertex expanders, and derived conditions on the expansion parameters and the local code that allow the algorithm to correct a linear number of adversarial errors in linear time. In particular, our results suggest that strong expansion from the constraint side can relax the required expansion from the variable side, and can improve the trade-off between the variable-side expansion ratio and the local distance compared with existing results for Tanner codes on one-sided expanders.

A natural open problem is to characterize the optimal trade-off between the expansion ratios and the local distance for linear-time decoding from a linear number of adversarial errors. This question remains open even for Tanner codes defined on one-sided expanders (and even for $d_0 = 2$) Indeed, it was conjectured in [She+25] that the condition $\delta_L d_0 > 1$ is both necessary and sufficient for Tanner codes defined on (c, d, α, δ_L) one-sided expanders to correct $\Omega(n)$ adversarial errors in $O(n)$ time. The necessity of this condition was established in [She+25]. Below, we give a simple construction showing that the sufficiency holds in a zero-rate setting.

Proposition 4.1. *Let $\mathcal{G} = (V \cup U, E)$ be a (c, d, α, δ_L) left expander, and let $\mathcal{C}_0$ be the binary repetition code of length d, so that $d_0 = d$. If $\delta_L d_0 > 1$, then there exists a decoding algorithm for $T(\mathcal{G}, \mathcal{C}_0)$ that corrects $\Omega(n)$ errors in $O(n)$ time.*

Proof. Since $d_0 = d$, the condition $\delta_L d_0 > 1$ is equivalent to $\delta_L > 1/d$. We first show that every connected component of $\mathcal{G}$ contains a linear number of variable nodes.

Let C be the vertex set of a connected component of $\mathcal{G}$, and define

$$X = C \cap V, \qquad Y = C \cap U.$$

Since $\mathcal{G}$ is (c, d)-regular, the number of edges in this component satisfies $c|X| = d|Y|$. Therefore, we obtain

$$|C| = |X| + |Y| = \left(1 + \frac{c}{d}\right)|X|.$$

It remains to show that $|X| = \Omega(n)$. Suppose, toward a contradiction, that $|X| \leq \alpha n$. By the definition of C, we have $N(X) = Y$. Thus, by left expansion,

$$|Y| = |N(X)| \geq \delta_L c|X|.$$

On the other hand, $c|X| = d|Y|$, so $|Y| = c|X|/d$. Therefore,

$$\frac{c}{d}|X| \geq \delta_L c|X|,$$

which implies $\delta_L \leq 1/d$, contradicting $\delta_L > 1/d$. Hence, $|X| > \alpha n$ for every connected component, and so each component contains $\Omega(n)$ variable nodes.

Now, since $\mathcal{C}_0$ is the binary repetition code, each local constraint forces all neighboring variable nodes to be equal. Consequently, all variable nodes within the same connected component must have the same value. Thus, $T(\mathcal{G}, \mathcal{C}_0)$ is a direct sum of binary repetition codes, one for each connected component, with lengths equal to the numbers of variable nodes in the corresponding components. Since every component has $\Omega(n)$ variable nodes, the minimum distance of $T(\mathcal{G}, \mathcal{C}_0)$ is $\Omega(n)$. Majority decoding on each component therefore corrects a number of errors up to half the minimum distance, which is $\Omega(n)$, and this decoding can clearly be implemented in $O(n)$ time. □

The Tanner codes in Proposition 4.1 have asymptotically zero rate, since the dimension is equal to the number of connected components, which is bounded by a constant depending only on α. Thus, while the proposition shows that the conjectured condition $\delta_L d_0 > 1$ is sufficient in a zero-rate regime, it leaves open the more interesting question of whether the same condition is necessary and sufficient for families of Tanner codes with non-vanishing asymptotic rate. More precisely, one may ask whether $\delta_L d_0 > 1$ characterizes the possibility of correcting $\Omega(n)$ errors in $O(n)$ time for positive-rate Tanner codes defined on (c, d, α, δ_L) one-sided expanders.

Another direction concerns the role of *spectral expansion*. Sipser and Spielman [SS96] showed that bipartite vertex expanders can be constructed explicitly from spectral expanders [Mar73, LPS88] via edge-vertex incidence graphs. Although this approach does not yield strong vertex-expansion parameters [Kah95], Tanner codes defined on such graphs can still have excellent error-correcting capabilities; see, for example, [SS96, Zém01, SR03]. In this setting, however, the decoding guarantees are governed primarily by the *spectral gap*, rather than directly by vertex-expansion ratios. This suggests that spectral properties of the underlying graph may sometimes provide stronger decoding guarantees than those obtainable from vertex expansion alone. It would be interesting to understand whether relatively weak vertex expansion, possibly together with additional structural assumptions, can imply useful spectral properties, and whether such properties can improve the known trade-offs among expansion, local distance, and decoding radius.

Finally, it would be interesting to extend the questions to non-binary Tanner codes. In particular, one may ask for analogous conditions under which Tanner codes with q-ary linear local codes admit linear-time decoding from a linear number of adversarial errors.

References

[Cap+02] M. Capalbo, O. Reingold, S. Vadhan, and A. Wigderson. "Randomness conductors and constant-degree lossless expanders". In: *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*. 2002, pp. 659–668. DOI: 10.1145/509907.510003.

[Chi+10] S. K. Chilappagari, D. V. Nguyen, B. Vasic, and M. W. Marcellin. "On trapping sets and guaranteed error correction capability of LDPC codes and GLDPC codes". In: *IEEE Transactions on Information Theory* 56.4 (2010), pp. 1600–1611. DOI: 10.1109/TIT.2010.2040962.

[CRT23] I. Cohen, R. Roth, and A. Ta-Shma. "HDX condensers". In: *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE. 2023, pp. 1649–1664. DOI: 10.1109/FOCS57990.2023.00100.

[DG17] M. Dowling and S. Gao. "Fast decoding of expander codes". In: *IEEE Transactions on Information Theory* 64.2 (2017), pp. 972–978. DOI: 10.1109/TIT.2017.2726064.

[Gol24] L. Golowich. "New explicit constant-degree lossless expanders". In: *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2024, pp. 4963–4971. DOI: 10.1137/1.9781611977912.177.

[HLW06] S. Hoory, N. Linial, and A. Wigderson. "Expander graphs and their applications". In: *Bulletin of the American Mathematical Society* 43.4 (2006), pp. 439–561. DOI: 10.1090/S0273-0979-06-01126-8.

[Hsi+25a] J.-T. Hsieh, T.-C. Lin, S. Mohanty, R. O'Donnell, and R. Y. Zhang. "Explicit two-sided vertex expanders beyond the spectral barrier". In: *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*. 2025, pp. 833–842. DOI: 10.1145/3717823.3718241.

[Hsi+25b] J.-T. Hsieh, A. Lubotzky, S. Mohanty, A. Reiner, and R. Y. Zhang. "Explicit lossless vertex expanders". In: *2025 IEEE 66th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE. 2025, pp. 894–911. DOI: 10.1109/FOCS63196.2025.00046.

[Kah95] N. Kahale. "Eigenvalues and expansion of regular graphs". In: *Journal of the ACM (JACM)* 42.5 (1995), pp. 1091–1106. DOI: 10.1145/210118.210136.

[LPS88] A. Lubotzky, R. Phillips, and P. Sarnak. "Ramanujan graphs". In: *Combinatorica* 8.3 (1988), pp. 261–277.

[Mar73] G. A. Margulis. "Explicit constructions of concentrators". In: *Problemy Peredachi Informatsii* 9.4 (1973), pp. 71–80.

[She+25] Y. Shen, C. Shangguan, M. Ouyang, and K. Cheng. "When can an expander code correct $\Omega(n)$ errors in $O(n)$ time?" In: *IEEE Transactions on Information Theory* 71.10 (2025), pp. 7626–7643. DOI: 10.1109/TIT.2025.3586842.

[SS96] M. Sipser and D. A. Spielman. "Expander codes". In: *IEEE Transactions on Information Theory* 42.6 (1996), pp. 1710–1722. DOI: 10.1109/18.556667.

[SR03] V. Skachek and R. M. Roth. "Generalized minimum distance iterative decoding of expander codes". In: *Proceedings 2003 IEEE Information Theory Workshop (ITW)*. IEEE. 2003, pp. 245–248. DOI: 10.1109/ITW.2003.1216740.

[Tan81] R. M. Tanner. "A recursive approach to low complexity codes". In: *IEEE Transactions on Information Theory* 27.5 (1981), pp. 533–547. DOI: 10.1109/TIT.1981.1056404.

[Vid13] M. Viderman. "Linear-time decoding of regular expander codes". In: *ACM Transactions on Computation Theory* 5.3 (2013), 10:1–10:25. DOI: 10.1145/2493252.2493255.

[Zém01] G. Zémor. "On expander codes". In: *IEEE Transactions on Information Theory* 47.2 (2001), pp. 835–837. DOI: 10.1109/18.910593.

[ZG25] Z. Zhou and Z. Guo. "Improved decoding of Tanner codes". In: *2025 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2025, pp. 1–6. DOI: 10.1109/ISIT63088.2025.11195639. arXiv: 2501.12293 [cs.IT].

Algorithm 1 PeelingDecoder

Require: Set $X \subseteq V$ and received word $\boldsymbol{y} \in \mathbb{F}_2^n$
Ensure: Codeword $\boldsymbol{x} \in T(\mathcal{G}, \mathcal{C}_0)$

for all $u \in U$ **do**
  $S(u) \leftarrow \emptyset$
end for
for all $v \in X$ **do**
  if $v \in N(u)$ **then**
    $S(u) \leftarrow S(u) \cup \{v\}$
  end if
end for
Initialize an empty queue Q
for all $u \in U$ **do**
  if $1 \leq |S(u)| \leq d_0 - 1$ **then**
    Enqueue u into Q
  end if
end for
$\boldsymbol{x} \leftarrow \boldsymbol{y}$
while Q is not empty **do**
  $u \leftarrow \mathsf{dequeue}(Q)$
  if $|S(u)| \geq 1$ **then**
    Mark $\boldsymbol{y}_{S(u)}$ as erasures, decode $\boldsymbol{y}_{N(u)}$ using a local erasure decoder of $\mathcal{C}_0$, and assign the decoded codeword to $\boldsymbol{x}_{N(u)}$
    $S \leftarrow S(u)$
    for all $v \in S$ **do**
      $X \leftarrow X \setminus \{v\}$
      for all $u' \in N(v)$ **do**
        $S(u') \leftarrow S(u') \setminus \{v\}$
        if $|S(u')| = d_0 - 1$ **then**
          Enqueue u' into Q
        end if
      end for
    end for
  end if
end while
return $\boldsymbol{x}$

Algorithm 2 Decoding Algorithm

Require: Received word $\boldsymbol{y} \in \mathbb{F}_2^n$ and parameter $\ell \in [d_0 - 1]$
Ensure: Codeword $\boldsymbol{x} \in T(\mathcal{G}, \mathcal{C}_0)$

$(\bar{\mathcal{E}}_\ell, \tilde{\boldsymbol{y}}) \leftarrow$ ErrorLocator$(\boldsymbol{y}, \ell)$
$\boldsymbol{x} \leftarrow$ PeelingDecoder$(\bar{\mathcal{E}}_\ell, \tilde{\boldsymbol{y}})$
return $\boldsymbol{x}$

Algorithm 3 FindCompletion

Require: Initial set $\mathcal{E} \subseteq V$, parameter $\ell \in [d_0 - 1]$, and collection of supports $\mathscr{T}$
Ensure: ℓ-completion set $\bar{\mathcal{E}}_\ell \subseteq V$

$X \leftarrow \mathcal{E}$
for all $v \in V$ **do**
    $\mathrm{inc}(v) \leftarrow \varnothing$
end for
Initialize an empty queue Q
for all $S \in \mathscr{T}$ **do**
    $\mathrm{absent}(S) \leftarrow |S \setminus X|$
    for all $v \in S$ **do**
        $\mathrm{inc}(v) \leftarrow \mathrm{inc}(v) \cup \{S\}$
    end for
    if $1 \leq \mathrm{absent}(S) \leq \ell$ **then**
        Enqueue S into Q
    end if
end for
while Q is not empty **do**
    $S \leftarrow \mathsf{dequeue}(Q)$
    if $\mathrm{absent}(S) \geq 1$ **then**
        for all $v \in S \setminus X$ **do**
            $X \leftarrow X \cup \{v\}$
            for all $S' \in \mathrm{inc}(v)$ **do**
                $\mathrm{absent}(S') \leftarrow \mathrm{absent}(S') - 1$
                if $\mathrm{absent}(S') = \ell$ **then**
                    Enqueue S' into Q
                end if
            end for
        end for
    end if
end while
$\bar{\mathcal{E}}_\ell \leftarrow X$
return $\bar{\mathcal{E}}_\ell$

Algorithm 4 ErrorLocator

Require: Received word $\boldsymbol{y} \in \mathbb{F}_2^n$ and parameter $\ell \in [d_0 - 1]$
Ensure: ℓ-completion set $\bar{\mathcal{E}}_\ell \subseteq V$ and partially decoded word $\tilde{\boldsymbol{y}} \in (\mathbb{F}_2 \cup \{\star\})^n$

for all $v \in V$ **do**
  $P_v \leftarrow \varnothing$
end for
for all $u \in U$ **do**
  $\boldsymbol{z}_u \leftarrow \mathcal{D}_0(\boldsymbol{y}_{N(u)})$
  for all $v \in N(u)$ **do**
    $P_v \leftarrow P_v \cup \{z_{u,v}\}$
  end for
end for
$\mathcal{E}^{(0)} \leftarrow \varnothing$
for all $v \in V$ **do**
  if $P_v = \{b\}$ for some $b \in \mathbb{F}_2$ **then**
    $\tilde{y}_v \leftarrow b$
  else
    $\tilde{y}_v \leftarrow \star$
    $\mathcal{E}^{(0)} \leftarrow \mathcal{E}^{(0)} \cup \{v\}$
  end if
end for
$\mathscr{T} \leftarrow \varnothing$
for all $u \in U$ **do**
  Write $N(u) = \{v_1^{(u)}, \ldots, v_d^{(u)}\}$ according to its fixed ordering
  $\mathscr{T}_u \leftarrow \{\mathrm{supp}_u(\boldsymbol{c}) \colon \boldsymbol{c} \in \mathcal{C}_0 \setminus \{0\}\}$
  $\mathscr{T} \leftarrow \mathscr{T} \cup \mathscr{T}_u$
end for
$\bar{\mathcal{E}}_\ell \leftarrow$ FindCompletion$(\mathcal{E}^{(0)}, \ell, \mathscr{T})$
$\tilde{\boldsymbol{y}} \leftarrow (\tilde{y}_v)_{v \in V}$
return $(\bar{\mathcal{E}}_\ell, \tilde{\boldsymbol{y}})$